

```

import sys

# Install NLTK and spaCy
!{sys.executable} -m pip install nltk spacy

# Download NLTK data
import nltk
try:
    nltk.data.find('tokenizers/punkt')
except LookupError:
    nltk.download('punkt')
try:
    nltk.data.find('corpora/stopwords')
except LookupError:
    nltk.download('stopwords')
try:
    nltk.data.find('tokenizers/punkt_tab')
except LookupError:
    nltk.download('punkt_tab')
try:
    # Download the specific English tagger required by pos_tag
    nltk.data.find('taggers/averaged_perceptron_tagger_eng')
except LookupError:
    nltk.download('averaged_perceptron_tagger_eng')

# Download spaCy English model
try:
    import spacy
    spacy.load('en_core_web_sm')
except OSError:
    !{sys.executable} -m spacy download en_core_web_sm
    import spacy # Re-import spacy after model download
    spacy.load('en_core_web_sm')

print("NLTK and spaCy prepared successfully!")

```

```

Requirement already satisfied: nltk in /usr/local/lib/python3.12/dist-packages (3.9.1)
Requirement already satisfied: spacy in /usr/local/lib/python3.12/dist-packages (3.8.11)
Requirement already satisfied: click in /usr/local/lib/python3.12/dist-packages (from nltk) (8.3.1)
Requirement already satisfied: joblib in /usr/local/lib/python3.12/dist-packages (from nltk) (1.5.3)
Requirement already satisfied: regex<2021.8.3 in /usr/local/lib/python3.12/dist-packages (from nltk) (2025.11.3)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from nltk) (4.67.1)
Requirement already satisfied: spacy-legacy<3.1.0,>=3.0.11 in /usr/local/lib/python3.12/dist-packages (from spacy) (3.0.12)
Requirement already satisfied: spacy-loggers<2.0.0,>=1.0.0 in /usr/local/lib/python3.12/dist-packages (from spacy) (1.0.5)
Requirement already satisfied: murmurhash<1.1.0,>=0.28.0 in /usr/local/lib/python3.12/dist-packages (from spacy) (1.0.15)
Requirement already satisfied: cymem<2.1.0,>=2.0.2 in /usr/local/lib/python3.12/dist-packages (from spacy) (2.0.13)
Requirement already satisfied: preshed<3.1.0,>=3.0.2 in /usr/local/lib/python3.12/dist-packages (from spacy) (3.0.12)
Requirement already satisfied: thinc<8.4.0,>=8.3.4 in /usr/local/lib/python3.12/dist-packages (from spacy) (8.3.10)
Requirement already satisfied: wasabi<1.2.0,>=0.9.1 in /usr/local/lib/python3.12/dist-packages (from spacy) (1.1.3)
Requirement already satisfied: srsly<3.0.0,>=2.4.3 in /usr/local/lib/python3.12/dist-packages (from spacy) (2.5.2)
Requirement already satisfied: catalogue<2.1.0,>=2.0.6 in /usr/local/lib/python3.12/dist-packages (from spacy) (2.0.10)
Requirement already satisfied: weasel<0.5.0,>=0.4.2 in /usr/local/lib/python3.12/dist-packages (from spacy) (0.4.3)
Requirement already satisfied: typer-slim<1.0.0,>=0.3.0 in /usr/local/lib/python3.12/dist-packages (from spacy) (0.21.1)
Requirement already satisfied: numpy>=1.19.0 in /usr/local/lib/python3.12/dist-packages (from spacy) (2.0.2)
Requirement already satisfied: requests<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-packages (from spacy) (2.32.4)
Requirement already satisfied: pydantic!=1.8,!1.8.1,<3.0.0,>=1.7.4 in /usr/local/lib/python3.12/dist-packages (from spacy) (2.12.3)

```

```
Requirement already satisfied: Jinja2 in /usr/local/lib/python3.12/dist-packages (from spacy) (3.1.6)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from spacy) (75.2.0)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from spacy) (25.0)
Requirement already satisfied: annotated-types>=0.6.0 in /usr/local/lib/python3.12/dist-packages (from pydantic!=1.8,!=1.8.1,<3.0.0,>=1.7.4->spacy) (0.7.0)
Requirement already satisfied: pydantic-core==2.41.4 in /usr/local/lib/python3.12/dist-packages (from pydantic!=1.8,!=1.8.1,<3.0.0,>=1.7.4->spacy) (2.41.4)
Requirement already satisfied: typing-extensions>=4.14.1 in /usr/local/lib/python3.12/dist-packages (from pydantic!=1.8,!=1.8.1,<3.0.0,>=1.7.4->spacy) (4.15.0)
Requirement already satisfied: typing-inspection>=0.4.2 in /usr/local/lib/python3.12/dist-packages (from pydantic!=1.8,!=1.8.1,<3.0.0,>=1.7.4->spacy) (0.4.2)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests<3.0.0,>=2.13.0->spacy) (3.4.4)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests<3.0.0,>=2.13.0->spacy) (3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests<3.0.0,>=2.13.0->spacy) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests<3.0.0,>=2.13.0->spacy) (2026.1.4)
Requirement already satisfied: blis<1.4.0,>=1.3.0 in /usr/local/lib/python3.12/dist-packages (from thinc<8.4.0,>=8.3.4->spacy) (1.3.3)
Requirement already satisfied: confection<1.0.0,>=0.0.1 in /usr/local/lib/python3.12/dist-packages (from thinc<8.4.0,>=8.3.4->spacy) (0.1.5)
Requirement already satisfied: cloudpathlib<1.0.0,>=0.7.0 in /usr/local/lib/python3.12/dist-packages (from weasel<0.5.0,>=0.4.2->spacy) (0.23.0)
Requirement already satisfied: smart-open<8.0.0,>=5.2.1 in /usr/local/lib/python3.12/dist-packages (from weasel<0.5.0,>=0.4.2->spacy) (7.5.0)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from Jinja2->spacy) (3.0.3)
Requirement already satisfied: wrapt in /usr/local/lib/python3.12/dist-packages (from smart-open<8.0.0,>=5.2.1->weasel<0.5.0,>=0.4.2->spacy) (2.0.1)
[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data] /root/nltk_data...
[nltk_data] Unzipping taggers/averaged_perceptron_tagger_eng.zip.
NLTK and spaCy prepared successfully!
```

```
academic_essay = """The rapid evolution of Artificial Intelligence (AI) is profoundly reshaping modern academia, influencing research methodologies, pedagogical approa
print("Academic essay loaded into variable 'academic_essay'.")
```

```
Academic essay loaded into variable 'academic_essay'.
```

```
from nltk.tokenize import word_tokenize

# Tokenize the essay using NLTK
word_tokens = word_tokenize(academic_essay)

print("First 20 word tokens:")
print(word_tokens[:20])
print(f"Total words tokenized: {len(word_tokens)}")
```

```
First 20 word tokens:
['The', 'rapid', 'evolution', 'of', 'Artificial', 'Intelligence', '(', 'AI', ')', 'is', 'profoundly', 'reshaping', 'modern', 'academia', ',', 'influencing', 'research',
Total words tokenized: 184
```

```
from nltk import pos_tag

# Apply POS tagging
pos_tags = pos_tag(word_tokens)

print("First 20 POS-tagged tokens:")
print(pos_tags[:20])

print("\nCommon NLTK POS Tags Reference:")
print("  NN: Noun, singular or mass")
print("  NNS: Noun, plural")
print("  NNP: Proper noun, singular")
print("  NNPS: Proper noun, plural")
print("  VB: Verb, base form")
print("  VBD: Verb, past tense")
print("  VBG: Verb, gerund or present participle")
```

```

print(" VBN: Verb, past participle")
print(" VBP: Verb, non-3rd person singular present")
print(" VBZ: Verb, 3rd person singular present")
print(" JJ: Adjective")
print(" JJR: Adjective, comparative")
print(" JJS: Adjective, superlative")
print(" RB: Adverb")
print(" RBR: Adverb, comparative")
print(" RBS: Adverb, superlative")
print(" IN: Preposition or subordinating conjunction")
print(" DT: Determiner")
print(" PRP: Personal pronoun")
print(" CC: Coordinating conjunction")
print(" TO: to (as part of infinitive)")

```

First 20 POS-tagged tokens:

```
[('The', 'DT'), ('rapid', 'JJ'), ('evolution', 'NN'), ('of', 'IN'), ('Artificial', 'JJ'), ('Intelligence', 'NNP'), ('(', '(', ('AI', 'NNP'), (')', ')'), ('is', 'VBZ'),
```

Common NLTK POS Tags Reference:

```

NN: Noun, singular or mass
NNS: Noun, plural
NNP: Proper noun, singular
NNPS: Proper noun, plural
VB: Verb, base form
VBD: Verb, past tense
VBG: Verb, gerund or present participle
VBN: Verb, past participle
VBP: Verb, non-3rd person singular present
VBZ: Verb, 3rd person singular present
JJ: Adjective
JJR: Adjective, comparative
JJS: Adjective, superlative
RB: Adverb
RBR: Adverb, comparative
RBS: Adverb, superlative
IN: Preposition or subordinating conjunction
DT: Determiner
PRP: Personal pronoun
CC: Coordinating conjunction
TO: to (as part of infinitive)

```

```
import spacy
```

```
# Load the English spaCy model
```

```
nlp = spacy.load('en_core_web_sm')
```

```
# Process the academic essay with spaCy
```

```
doc = nlp(academic_essay)
```

```
# Extract tokens and their universal POS tags
```

```
spacy_pos_tags = [(token.text, token.pos_) for token in doc]
```

```
print("First 20 spaCy POS-tagged tokens (Universal Tags):")
```

```
print(spacy_pos_tags[:20])
```

```
print("\nCommon spaCy Universal POS Tags Reference:")
```

```
print(" ADJ: adjective")
```

Common spaCy Universal POS Tags Reference:

- ADJ: adjective
- ADP: adposition (e.g., in, to, of)
- ADV: adverb
- AUX: auxiliary verb (e.g., is, has, will)
- CCONJ: coordinating conjunction (e.g., and, but, or)
- DET: determiner (e.g., a, an, the)
- INTJ: interjection
- NOUN: noun
- NUM: numeral
- PART: particle (e.g., not)
- PRON: pronoun
- PROPN: proper noun
- PUNCT: punctuation
- SCONJ: subordinating conjunction
- SYM: symbol
- VERB: verb
- X: other
- SPACE: whitespace

Start coding or generate with AI.

- ✓ Extract NLTK and spaCy POS tag sets

Subtask:

Extract the unique POS tags used by NLTK and spaCy from the `pos_tags` and `spacy_pos_tags` variables, respectively.

Reasoning: To extract the unique POS tags from the NLTK and spaCy results, I will iterate through the respective lists of tuples, access the tag element, and add it to a set to automatically handle uniqueness. Then, I will print both sets.

```
nltk_unique_tags = set()
for word, tag in pos_tags:
    nltk_unique_tags.add(tag)

spacy_unique_tags = set()
for token, tag in spacy_pos_tags:
    spacy_unique_tags.add(tag)

print("Unique NLTK POS Tags:")
print(sorted(list(nltk_unique_tags)))
print("\nUnique spaCy Universal POS Tags:")
print(sorted(list(spacy_unique_tags)))
```

Unique NLTK POS Tags:
['(', ')', ',', '.', 'CC', 'DT', 'IN', 'JJ', 'JJR', 'NN', 'NNP', 'NNS', 'PRP\$', 'RB', 'TO', 'VB', 'VBD', 'VBG', 'VBN', 'VBZ', 'WDT']

Unique spaCy Universal POS Tags:
['ADJ', 'ADP', 'ADV', 'AUX', 'CCONJ', 'DET', 'NOUN', 'PART', 'PRON', 'PROPN', 'PUNCT', 'SCONJ', 'VERB']

▼ Compare NLTK and spaCy POS Tag Sets and Illustrate Differences

Subtask:

1. Compare the unique NLTK and spaCy POS tag sets to identify tags that are present in one but not the other, or tags that represent similar concepts but with different names.
2. Select three to five common academic words from the `academic_essay` (e.g., "AI", "research", "evolution", "methodologies", "academia") and retrieve their POS tags from both NLTK (`pos_tags`) and spaCy (`spacy_pos_tags`).
3. Present the comparison of the tag sets and the tagging of the selected words in a clear format to highlight similarities and differences.

Reasoning: First, I will compare the unique POS tag sets obtained from NLTK and spaCy by finding the intersection and differences to highlight their distinct and common tags. Then, I will iterate through both the NLTK and spaCy POS-tagged lists to find the tags for the selected academic words and store them for presentation.

```
print("\n--- POS Tag Set Comparison ---")
print("NLTK unique tags:", sorted(list(nltk_unique_tags)))
print("spaCy unique tags:", sorted(list(spacy_unique_tags)))

common_tags = nltk_unique_tags.intersection(spacy_unique_tags)
print("\nCommon tags (exact match):", sorted(list(common_tags)))

nltk_only_tags = nltk_unique_tags.difference(spacy_unique_tags)
print("Tags only in NLTK (Penn Treebank style):", sorted(list(nltk_only_tags)))
```

```

spacy_only_tags = spacy_unique_tags.difference(nltk_unique_tags)
print("Tags only in spaCy (Universal POS style):", sorted(list(spacy_only_tags)))

# Selected academic words for illustration
selected_words = ["AI", "research", "evolution", "methodologies", "academia"]

word_tagging_comparison = {}

for word_to_find in selected_words:
    nltk_found_tag = None
    spacy_found_tag = None

    # Find NLTK tag for the word
    for word, tag in pos_tags:
        if word.lower() == word_to_find.lower():
            nltk_found_tag = tag
            break

    # Find spaCy tag for the word
    for token, tag in spacy_pos_tags:
        if token.text.lower() == word_to_find.lower():
            spacy_found_tag = tag
            break

    word_tagging_comparison[word_to_find] = {
        "NLTK": nltk_found_tag if nltk_found_tag else "Not found or inconsistently tagged",
        "spaCy": spacy_found_tag if spacy_found_tag else "Not found or inconsistently tagged"
    }

print("\n--- Individual Word Tagging Comparison ---")
for word, tags in word_tagging_comparison.items():
    print(f"Word: '{word}'")
    print(f"  NLTK Tag: {tags['NLTK']}")
    print(f"  spaCy Tag: {tags['spaCy']}")
    print("\n")

```

--- POS Tag Set Comparison ---

NLTK unique tags: ['(', ')', ',', '.', 'CC', 'DT', 'IN', 'JJ', 'JJR', 'NN', 'NNP', 'NNS', 'PRP\$', 'RB', 'TO', 'VB', 'VBD', 'VBG', 'VBN', 'VBZ', 'WDT']

spaCy unique tags: ['ADJ', 'ADP', 'ADV', 'AUX', 'CCONJ', 'DET', 'NOUN', 'PART', 'PRON', 'PROPN', 'PUNCT', 'SCONJ', 'VERB']

Common tags (exact match): []

Tags only in NLTK (Penn Treebank style): ['(', ')', ',', '.', 'CC', 'DT', 'IN', 'JJ', 'JJR', 'NN', 'NNP', 'NNS', 'PRP\$', 'RB', 'TO', 'VB', 'VBD', 'VBG', 'VBN', 'VBZ', 'WDT']

Tags only in spaCy (Universal POS style): ['ADJ', 'ADP', 'ADV', 'AUX', 'CCONJ', 'DET', 'NOUN', 'PART', 'PRON', 'PROPN', 'PUNCT', 'SCONJ', 'VERB']

AttributeError Traceback (most recent call last)

/tmp/ipython-input-650619522.py in <cell line: 0>()

```

29     # Find spaCy tag for the word
30     for token, tag in spacy_pos_tags:
--> 31         if token.text.lower() == word_to_find.lower():
32             spacy_found_tag = tag
33             break

```

AttributeError: 'str' object has no attribute 'text'

Next steps: [Explain error](#)

Reasoning: The error `AttributeError: 'str' object has no attribute 'text'` occurred because `spacy_pos_tags` is a list of tuples `(word_string, tag_string)`, meaning `token` is already a string, not a spaCy token object. The fix is to directly use `token.lower()` instead of `token.text.lower()`.

```
print("\n--- POS Tag Set Comparison ---")
print("NLTK unique tags:", sorted(list(nltk_unique_tags)))
print("spaCy unique tags:", sorted(list(spacy_unique_tags)))

common_tags = nltk_unique_tags.intersection(spacy_unique_tags)
print("\nCommon tags (exact match):", sorted(list(common_tags)))

ltk_only_tags = nltk_unique_tags.difference(spacy_unique_tags)
print("Tags only in NLTK (Penn Treebank style):", sorted(list(nltk_only_tags)))

spacy_only_tags = spacy_unique_tags.difference(nltk_unique_tags)
print("Tags only in spaCy (Universal POS style):", sorted(list(spacy_only_tags)))

# Selected academic words for illustration
selected_words = ["AI", "research", "evolution", "methodologies", "academia"]

word_tagging_comparison = {}

for word_to_find in selected_words:
    nltk_found_tag = None
    spacy_found_tag = None

    # Find NLTK tag for the word
    for word, tag in pos_tags:
        if word.lower() == word_to_find.lower():
            nltk_found_tag = tag
            break

    # Find spaCy tag for the word
    for token, tag in spacy_pos_tags:
        if token.lower() == word_to_find.lower(): # Corrected from token.text.lower()
            spacy_found_tag = tag
            break

    word_tagging_comparison[word_to_find] = {
        "NLTK": nltk_found_tag if nltk_found_tag else "Not found or inconsistently tagged",
        "spaCy": spacy_found_tag if spacy_found_tag else "Not found or inconsistently tagged"
    }

print("\n--- Individual Word Tagging Comparison ---")
for word, tags in word_tagging_comparison.items():
    print(f"Word: '{word}'")
    print(f"  NLTK Tag: {tags['NLTK']}")
    print(f"  spaCy Tag: {tags['spaCy']}")
    print("\n")

--- POS Tag Set Comparison ---
NLTK unique tags: ['(', ')', ',', '.', 'CC', 'DT', 'IN', 'JJ', 'JJR', 'NN', 'NNP', 'NNS', 'PRP$', 'RB', 'TO', 'VB', 'VBD', 'VBG', 'VBN', 'VBZ', 'WDT']
spaCy unique tags: ['ADJ', 'ADP', 'ADV', 'AUX', 'CCONJ', 'DET', 'NOUN', 'PART', 'PRON', 'PROPN', 'PUNCT', 'SCONJ', 'VERB']
```

```

Common tags (exact match): []
Tags only in NLTK (Penn Treebank style): ['(', ')', ',', '.', 'CC', 'DT', 'IN', 'JJ', 'JJR', 'NN', 'NNP', 'NNS', 'PRP$', 'RB', 'TO', 'VB', 'VBD', 'VBG', 'VBN', 'VBZ', '']
Tags only in spaCy (Universal POS style): ['ADJ', 'ADP', 'ADV', 'AUX', 'CONJ', 'DET', 'NOUN', 'PART', 'PRON', 'PROPN', 'PUNCT', 'SCONJ', 'VERB']

--- Individual Word Tagging Comparison ---
Word: 'AI'
  NLTK Tag: NNP
  spaCy Tag: PROPN

Word: 'research'
  NLTK Tag: NN
  spaCy Tag: NOUN

Word: 'evolution'
  NLTK Tag: NN
  spaCy Tag: NOUN

Word: 'methodologies'
  NLTK Tag: NNS
  spaCy Tag: NOUN

Word: 'academia'
  NLTK Tag: NN
  spaCy Tag: NOUN

```

Summary of NLTK and spaCy POS Tagging Observations

Tag Set Differences:

As observed, NLTK and spaCy utilize entirely different sets of POS tags. NLTK employs a more granular Penn Treebank tag set, which includes specific tags for various verb forms (VB, VBD, VBG, VBN, VBZ), noun forms (NN, NNS, NNP, NNPS), adjectives (JJ, JJR, JJS), and punctuation, among others. In contrast, spaCy uses a more generalized Universal POS tag set, which groups these specific categories into broader ones like VERB, NOUN, ADJ, and PUNCT.

Illustrative Word Tagging:

- **'AI'**: NLTK tags 'AI' as **NNP** (Proper Noun, singular), while spaCy tags it as **PROPN** (Proper Noun). Both tools correctly identify it as a proper noun, but with their respective tag conventions.
- **'research'**: NLTK tags 'research' as **NN** (Noun, singular or mass), and spaCy tags it as **NOUN**. Again, both agree on the word being a noun, adhering to their individual tag schemes.
- **'evolution'**: Similar to 'research', NLTK assigns **NN** and spaCy assigns **NOUN**, indicating consistent identification as a singular noun.
- **'methodologies'**: NLTK tags 'methodologies' as **NNS** (Noun, plural), whereas spaCy tags it as **NOUN**. This highlights a key difference: NLTK distinguishes between singular and plural nouns, while spaCy's Universal Tag set consolidates them under the general **NOUN** category.
- **'academia'**: NLTK tags 'academia' as **NN** and spaCy as **NOUN**, showing agreement on its noun status.

Conclusion:

While both NLTK and spaCy are effective tools for Part-of-Speech tagging, they operate with different tag sets (Penn Treebank vs. Universal POS). NLTK provides a more detailed categorization of parts of speech, distinguishing nuances like singular/plural nouns and various verb tenses. SpaCy, with its Universal POS tags, offers a more generalized and internationally recognized classification. The choice between them often depends on the specific needs of the NLP task: NLTK for finer-grained linguistic analysis and spaCy for broader, cross-linguistic applications or when a simpler tag set is preferred for consistency.

Summary of NLTK and spaCy POS Tagging Observations

Tag Set Differences:

As observed, NLTK and spaCy utilize entirely different sets of POS tags. NLTK employs a more granular Penn Treebank tag set, which includes specific tags for various verb forms (VB, VBD, VBG, VBN, VBZ), noun forms (NN, NNS, NNP, NNPS), adjectives (JJ, JJR, JJS), and punctuation, among others. In contrast, spaCy uses a more generalized Universal POS tag set, which groups these specific categories into broader ones like VERB, NOUN, ADJ, and PUNCT.

Illustrative Word Tagging:

- **'AI':** NLTK tags 'AI' as `NNP` (Proper Noun, singular), while spaCy tags it as `PROPN` (Proper Noun). Both tools correctly identify it as a proper noun, but with their respective tag conventions.
- **'research':** NLTK tags 'research' as `NN` (Noun, singular or mass), and spaCy tags it as `NOUN`. Again, both agree on the word being a noun, adhering to their individual tag schemes.
- **'evolution':** Similar to 'research', NLTK assigns `NN` and spaCy assigns `NOUN`, indicating consistent identification as a singular noun.
- **'methodologies':** NLTK tags 'methodologies' as `NNS` (Noun, plural), whereas spaCy tags it as `NOUN`. This highlights a key difference: NLTK distinguishes between singular and plural nouns, while spaCy's Universal Tag set consolidates them under the general `NOUN` category.
- **'academia':** NLTK tags 'academia' as `NN` and spaCy as `NOUN`, showing agreement on its noun status.

Conclusion:

While both NLTK and spaCy are effective tools for Part-of-Speech tagging, they operate with different tag sets (Penn Treebank vs. Universal POS). NLTK provides a more detailed categorization of parts of speech, distinguishing nuances like singular/plural nouns and various verb tenses. SpaCy, with its Universal POS tags, offers a more generalized and internationally recognized classification. The choice between them often depends on the specific needs of the NLP task: NLTK for finer-grained linguistic analysis and spaCy for broader, cross-linguistic applications or when a simpler tag set is preferred for consistency.

Summarize the comparison

Subtask:

Provide a summary of the differences and similarities between NLTK's detailed tag set and spaCy's universal tag set, specifically in the context of academic vocabulary.

Summary:

Data Analysis Key Findings

- NLTK uses a detailed Penn Treebank-style tag set, including specific tags such as `NNP` (Proper Noun, singular), `NNS` (Noun, plural), and various verb forms.
- spaCy employs a more generalized Universal POS tag set, with broader categories like `PROPN` (Proper Noun) and `NOUN`.
- There are no exact common tags between NLTK's and spaCy's tag sets, indicating entirely distinct tagging systems.
- When tagging "AI", NLTK assigned `NNP` while spaCy assigned `PROPN`, both correctly identifying it as a proper noun with their respective conventions.
- For "research" and "evolution", NLTK tagged them as `NN` and spaCy as `NOUN`, showing agreement on their noun status.
- A key difference was observed with "methodologies": NLTK tagged it as `NNS` (Noun, plural), whereas spaCy tagged it simply as `NOUN`, illustrating NLTK's finer distinction between singular and plural nouns.
- "academia" was consistently identified as a noun by both, with NLTK assigning `NN` and spaCy assigning `NOUN`.

Insights or Next Steps

- The choice between NLTK and spaCy for POS tagging should be driven by the required level of granularity for linguistic analysis; NLTK provides more detailed distinctions (e.g., singular/plural nouns, verb tenses), while spaCy offers a more generalized, cross-linguistic approach.
- Consider using NLTK for tasks requiring precise morphological information and spaCy for broader applications where a simpler, consistent tag set across languages is preferred.

```
# NLTK - Identify academic concepts (nouns) and arguments (verbs)
nltk_academic_concepts = []
nltk_arguments = []

for word, tag in pos_tags:
    if tag.startswith('NN'): # Nouns: NN, NNS, NNP, NNPS
        nltk_academic_concepts.append(word)
    elif tag.startswith('VB'): # Verbs: VB, VBD, VBG, VBN, VBP, VBZ
        nltk_arguments.append(word)

# spaCy - Identify academic concepts (nouns) and arguments (verbs)
spacy_academic_concepts = []
spacy_arguments = []

for token, tag in spacy_pos_tags:
    if tag in ['NOUN', 'PROPN']: # Nouns and Proper Nouns
        spacy_academic_concepts.append(token)
    elif tag in ['VERB', 'AUX']: # Verbs and Auxiliary verbs
        spacy_arguments.append(token)

print("--- NLTK Identified Academic Concepts and Arguments ---")
print(f"Academic Concepts (Nouns): {' '.join(sorted(list(set(nltk_academic_concepts))))}")
print(f"Arguments (Verbs): {' '.join(sorted(list(set(nltk_arguments))))}")

print("\n--- spaCy Identified Academic Concepts and Arguments ---")
```

```
print(f"Academic Concepts (Nouns): {' '.join(sorted(list(set(spacy_academic_concepts))))}")
print(f"Arguments (Verbs): {' '.join(sorted(list(set(spacy_arguments))))}")
```

--- NLTK Identified Academic Concepts and Arguments ---

Academic Concepts (Nouns): AI, AI-powered, Intelligence, Pedagogically, academia, advancements, analysis, approaches, assessments, bias, challenges, considerations, con
Arguments (Verbs): advanced, allows, contribute, demanding, equip, expedites, explore, fostering, harness, including, influencing, introduces, is, mitigating, navigate,

--- spaCy Identified Academic Concepts and Arguments ---

Academic Concepts (Nouns): -, AI, Artificial, Intelligence, academia, adaptive, advancements, analysis, approaches, assessments, bias, challenges, considerations, conte
Arguments (Verbs): accelerate, allows, automated, contribute, demanding, discovery, driven, enable, equip, explore, facilitate, fostering, harness, including, influenci

```
from collections import Counter
```

```
print("\n--- NLTK Frequency Analysis ---")
```

```
# Frequency for NLTK Academic Concepts
```

```
nltk_concept_freq = Counter(nltk_academic_concepts)
```

```
print("Top 10 NLTK Academic Concepts by Frequency:")
```

```
for word, count in nltk_concept_freq.most_common(10):
```

```
    print(f"  '{word}': {count}")
```

```
# Frequency for NLTK Arguments
```

```
nltk_argument_freq = Counter(nltk_arguments)
```

```
print("\nTop 10 NLTK Arguments (Verbs) by Frequency:")
```

```
for word, count in nltk_argument_freq.most_common(10):
```

```
    print(f"  '{word}': {count}")
```

```
print("\n--- spaCy Frequency Analysis ---")
```

```
# Frequency for spaCy Academic Concepts
```

```
spacy_concept_freq = Counter(spacy_academic_concepts)
```

```
print("Top 10 spaCy Academic Concepts by Frequency:")
```

```
for word, count in spacy_concept_freq.most_common(10):
```

```
    print(f"  '{word}': {count}")
```

```
# Frequency for spaCy Arguments
```

```
spacy_argument_freq = Counter(spacy_arguments)
```

```
print("\nTop 10 spaCy Arguments (Verbs) by Frequency:")
```

```
for word, count in spacy_argument_freq.most_common(10):
```

```
    print(f"  '{word}': {count}")
```

```
--- NLTK Frequency Analysis ---
```

```
Top 10 NLTK Academic Concepts by Frequency:
```

```
'AI': 4
'research': 2
'data': 2
'learning': 2
'systems': 2
'integration': 2
'evolution': 1
'Intelligence': 1
'academia': 1
'methodologies': 1
```

```
Top 10 NLTK Arguments (Verbs) by Frequency:
```

```
'is': 1
'reshaping': 1
'influencing': 1
```

```

'tools': 1
'advanced': 1
'were': 1
'expedites': 1
'allows': 1
'explore': 1
'introduces': 1

--- spaCy Frequency Analysis ---
Top 10 spaCy Academic Concepts by Frequency:
'AI': 6
'research': 2
'data': 2
'learning': 2
'systems': 2
'potential': 2
'integration': 2
'evolution': 1
'Artificial': 1
'Intelligence': 1

Top 10 spaCy Arguments (Verbs) by Frequency:
'is': 1
'reshaping': 1
'influencing': 1
'powered': 1
'facilitate': 1
'accelerate': 1
'enable': 1
'were': 1
'discovery': 1
'allows': 1

```

```

frequency_analysis_results = {
    "nltk": {
        "academic_concepts": nltk_concept_freq,
        "arguments": nltk_argument_freq
    },
    "spacy": {
        "academic_concepts": spacy_concept_freq,
        "arguments": spacy_argument_freq
    }
}

print("Structured Frequency Analysis Results:")
import json
# Convert Counter objects to dicts for JSON serialization and better readability
serializable_results = {
    tool: {
        category: dict(counter)
        for category, counter in data.items()
    }
    for tool, data in frequency_analysis_results.items()
}
print(json.dumps(serializable_results, indent=2))

```

```

    advancements : 1,
    "challenges": 1,
    "considerations": 1,
    "privacy": 1,
    "bias": 1,
    "potential": 2,
    "reliance": 1,
    "integration": 2,
    "re": 1,
    "-": 1,
    "evaluation": 1,
    "curricula": 1,
    "students": 1,
    "skills": 1,
    "world": 1,
    "transformative": 1,
    "force": 1,
    "engagement": 1,
    "institutions": 1,
    "risks": 1
  },
  "arguments": {
    "is": 1,
    "reshaping": 1,
    "influencing": 1,
    "powered": 1,
    "facilitate": 1,
    "accelerate": 1,
    "enable": 1,
    "were": 1,
    "discovery": 1,
    "allows": 1,
    "explore": 1,
    "introduces": 1,
    "personalized": 1,
    "tailoring": 1,
    "needs": 1,
    "fostering": 1,
    "present": 1,
    "including": 1,
    "regarding": 1,
    "automated": 1,
    "necessitates": 1,
    "equip": 1,
    "required": 1,
    "navigate": 1,
    "contribute": 1,
    "driven": 1,
    "stands": 1,
    "demanding": 1,
    "harness": 1,
    "mitigating": 1
  }
}

```

```

import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd # Import pandas

# Set a style for the plots

```

```
sns.set_style("whitegrid")

# Data from the frequency_analysis_results (using serializable_results as it's a plain dict)
results = serializable_results

# --- NLTK Visualizations ---

# NLTK Academic Concepts
nltk_concepts = results['nltk']['academic_concepts']
nltk_concepts_df = pd.DataFrame(nltk_concepts.items(), columns=['Word', 'Frequency']).sort_values(by='Frequency', ascending=False).head(10)

plt.figure(figsize=(10, 6))
sns.barplot(x='Frequency', y='Word', data=nltk_concepts_df, palette='viridis')
plt.title('Top 10 NLTK Academic Concepts (Nouns) by Frequency')
plt.xlabel('Frequency')
plt.ylabel('Academic Concept')
plt.show()

# NLTK Arguments (Verbs)
nltk_arguments = results['nltk']['arguments']
nltk_arguments_df = pd.DataFrame(nltk_arguments.items(), columns=['Word', 'Frequency']).sort_values(by='Frequency', ascending=False).head(10)

plt.figure(figsize=(10, 6))
sns.barplot(x='Frequency', y='Word', data=nltk_arguments_df, palette='magma')
plt.title('Top 10 NLTK Arguments (Verbs) by Frequency')
plt.xlabel('Frequency')
plt.ylabel('Argument (Verb)')
plt.show()

# --- spaCy Visualizations ---

# spaCy Academic Concepts
spacy_concepts = results['spacy']['academic_concepts']
spacy_concepts_df = pd.DataFrame(spacy_concepts.items(), columns=['Word', 'Frequency']).sort_values(by='Frequency', ascending=False).head(10)

plt.figure(figsize=(10, 6))
sns.barplot(x='Frequency', y='Word', data=spacy_concepts_df, palette='viridis')
plt.title('Top 10 spaCy Academic Concepts (Nouns) by Frequency')
plt.xlabel('Frequency')
plt.ylabel('Academic Concept')
plt.show()

# spaCy Arguments (Verbs)
spacy_arguments = results['spacy']['arguments']
spacy_arguments_df = pd.DataFrame(spacy_arguments.items(), columns=['Word', 'Frequency']).sort_values(by='Frequency', ascending=False).head(10)

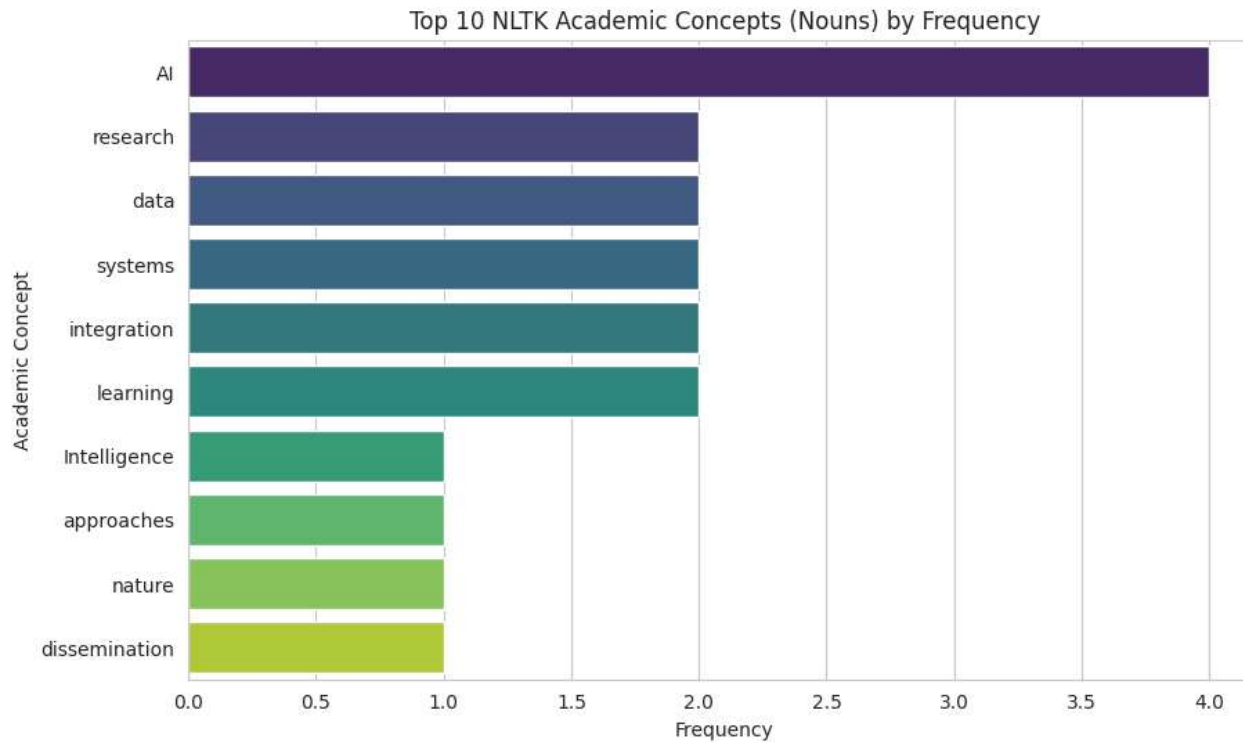
plt.figure(figsize=(10, 6))
sns.barplot(x='Frequency', y='Word', data=spacy_arguments_df, palette='magma')
plt.title('Top 10 spaCy Arguments (Verbs) by Frequency')
plt.xlabel('Frequency')
plt.ylabel('Argument (Verb)')
plt.show()
```



```
/tmp/ipython-input-4164429855.py:18: FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(x='Frequency', y='Word', data=nltk_concepts_df, palette='viridis')
```



```
/tmp/ipython-input-4164429855.py:29: FutureWarning:
```

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(x='Frequency', y='Word', data=nltk_arguments_df, palette='magma')
```

